Complete System Design Guide: From Basics to Advanced

A step-by-step guide starting with fundamental concepts, technologies, and frameworks, then progressing to real-world system design scenarios.

Table of Contents

PART 1: FUNDAMENTALS & BASIC CONCEPTS

Basic Concepts

1. What is a Server?

Simple Explanation: A server is a computer program (or the physical machine running it) that waits for requests from clients and sends back responses.

```
Think of it like a restaurant:
- Customer (Client) = You ordering food
- Server (Waiter) = Takes your order
- Kitchen (Server Application) = Prepares food
- Response = Food delivered to your table
```

Real Example: When you visit `google.com`:

1. Your browser (client) sends a request to Google's servers
2. Google's servers process your search query
3. Results come back to your browser

---

2. What is a Database?

Simple Explanation: A database is an organized collection of data stored in a computer that you can search, update, and retrieve.

```
Think of it like a library:
- Books = Data
- Shelves = Database tables
- Library system = Database management system
- Librarian = Database software
```

Two Main Types:

SQL Databases (Relational)

- Data organized in rows and columns (like Excel spreadsheet)
- Example: MySQL, PostgreSQL

- Best for: Structured data (user accounts, orders, transactions)

```
Users Table:
+--------+----------+----------------+
| UserID | Name     | Email          |
+--------+----------+----------------+
| 1      | John     | john@email.com |
| 2      | Sarah    | sarah@email.com|
+--------+----------+----------------+
```

NoSQL Databases (Non-Relational)

- Data stored as documents or key-value pairs
- Example: MongoDB, Cassandra
- Best for: Unstructured data (social media posts, images, flexible schemas)

```
MongoDB Document:
{
  "_id": 1,
  "name": "John",
  "email": "john@email.com",
  "hobbies": ["reading", "gaming"],
  "address": {
    "city": "New York",
    "zip": "10001"
  }
}
```

Simple Comparison:

```
SQL: Use when data has relationships and structure
NoSQL: Use when data is flexible and can vary
```

---

3. What is an API (Application Programming Interface)?

Simple Explanation: An API is a way for different programs to talk to each other and request/send information.

```
Think of it like a restaurant menu:
- You (Client) = Hungry customer
- Menu = API documentation (what's available)
```

```
- Waiter = API endpoint
- Your order = API request
- Delivered food = API response
```

Real Example:

```
When you check weather on your phone:

Your Phone → Sends request to Weather API
          → "Give me weather for New York"

Weather API → Processes request
            → Looks up data
            → Sends back JSON response

{
  "city": "New York",
  "temperature": 72,
  "humidity": 65,
  "condition": "Sunny"
}
```

REST API (Most Common Type)

- Uses HTTP methods: GET, POST, PUT, DELETE
- GET = Retrieve data
- POST = Create new data
- PUT = Update existing data
- DELETE = Remove data

```
Examples:
GET  /users/123          → Get user with ID 123
POST /users              → Create new user
PUT  /users/123          → Update user 123
DELETE /users/123        → Delete user 123
```

---

4. What is Scaling?

Simple Explanation: When your application gets more users, you need to handle more requests. Scaling means making your system capable of handling more load.

```
Analogy: Restaurant Growing

Small Restaurant:
- 1 waiter
- 1 table
- Can serve 10 customers/hour

Growing Business:
- Hire 5 waiters (Horizontal Scaling - add more)
- Get a bigger building (Vertical Scaling - upgrade)
- Open 3 more locations (Distributed Scaling)
```

Two Types:

Vertical Scaling (Scale-Up)

- Add more power to single machine
- More CPU, RAM, Storage
- Like upgrading a laptop with better processor

```
Before: 1 server with 4GB RAM
After:  1 server with 64GB RAM

Pros: Simple
Cons: Limited by single machine capacity ($$$)
```

Horizontal Scaling (Scale-Out)

- Add more machines to handle load
- Like adding more servers in data center

```
Before: 1 server handling 1000 users
After:  10 servers, each handling 100 users

Pros: Can scale infinitely
Cons: More complex coordination
```

---

5. What is Latency?

Simple Explanation: Latency is how long it takes for a request to be processed and get a response.

```
Think of it like a phone call:
- You speak (Request)
- Sound travels through network (Processing)
- Friend hears and responds (Response)
- Total time = Latency

Lower latency = Faster response
Higher latency = Slower response
```

Real Numbers:

```
Good latency:    < 100 milliseconds
Acceptable:      100-300 milliseconds
Poor:            > 1000 milliseconds (1 second)
```

---

6. What is Throughput?

Simple Explanation: Throughput is how many requests a system can handle per second.

```
Think of it like a highway:
- 1 lane highway = 100 cars/hour (low throughput)
- 10 lane highway = 1000 cars/hour (high throughput)

Throughput = Number of requests processed per second
             (also called QPS or RPS)
```

---

7. What is Availability?

Simple Explanation: Availability is the percentage of time your system is working without downtime.

```
99% availability = System is up 99% of time, down 1%
99.9% availability = System is down only 9 hours per year
99.99% availability = System is down only 52 minutes per year

Higher = Better (and more expensive)
```

---

8. What is Redundancy?

Simple Explanation: Redundancy means having backup copies of important things so if one fails, another takes over.

```
Think of it like backup seatbelts in a car:
- If primary seatbelt fails, backup activates
- Users don't even notice

In systems:
- Multiple servers instead of 1
- If server 1 dies, server 2 takes over
- Backups of database in different locations
```

---

9. What is Consistency?

Simple Explanation: Consistency means all copies of data are the same and up-to-date.

```
Imagine a bank:
- You deposit $100
- Your account shows $100 (consistent)
- ATM shows $100 (consistent)
- Everywhere you check = same amount

If ATM shows $100 but account shows $50 = Inconsistent
```

Two Types:

Strong Consistency (Immediate)

- All updates visible immediately everywhere
- Like updating in real-time
- Slower but more reliable

Eventual Consistency (Delayed)

- Updates take time to spread
- Like sending postcards through mail
- Faster but might see old data temporarily

---

10. What is a Transaction?

Simple Explanation: A transaction is a series of operations that either all succeed or all fail together.

```
Think of transferring money:
1. Subtract $100 from Account A
2. Add $100 to Account B

If step 1 succeeds but step 2 fails:
- Money disappears!
- Transaction rolls back (undoes both)
- Either both happen or neither happens
```

ACID Properties:

```
A - Atomicity: All or nothing
C - Consistency: Data stays valid
I - Isolation: Doesn't affect other transactions
D - Durability: Saved permanently
```

Core Technologies Explained

1. Redis (In-Memory Cache)

What is it? Redis is a super-fast storage system that keeps frequently accessed data in RAM (memory) instead of disk, for instant retrieval.

```
Think of it like:
- Regular Database = Library (search takes time)
- Redis = Your bookshelf (instant access)

Redis is 1000x faster than database for reads!
```

Real Analogy:

```
Restaurant Example:
- Database = Kitchen storage room (far away, hard to access)
- Redis = Prep station (close by, instant access)
- Popular orders cached in Redis
- Rare orders fetched from storage
```

Common Uses:

```
1. Session Storage
   - User logged in? Check Redis (instant)
   - Instead of querying database (slow)

2. Caching Database Results
   - Query executed? Store result in Redis
   - Next same query returns from Redis

3. Leaderboards & Counters
   - Game scores
   - View counts
   - Real-time rankings

4. Real-time Analytics
   - Count active users
   - Track trending topics
```

Data Structures in Redis:

```
String: "username" = "john"
List: [1, 2, 3, 4, 5]
Set: {apple, banana, orange} (no duplicates)
Hash: {"name": "John", "age": 30}
Sorted Set: {user1: 100, user2: 95, user3: 88} (ranked)
```

Example:

```
# Cache a user's profile
redis.set("user:123", {name: "John", email: "john@email.com"}, expire_in: 1_hour)

# Retrieve it instantly
user = redis.get("user:123")  # < 1ms response time

# Increment view counter
redis.incr("post:456:views")  # Atomic counter

# Ranking (leaderboard)
redis.zadd("leaderboard", 100, "player1")
redis.zadd("leaderboard", 95, "player2")
top_10 = redis.zrange("leaderboard", 0, 9, WITHSCORES)
```

Pros:

- Extremely fast (< 1ms)

- Simple to use
- Great for frequently accessed data

Cons:

- Data in memory (lost on restart if not persisted)
- Limited by RAM size
- Not ideal for permanent storage

---

2. PostgreSQL (SQL Database)

What is it? PostgreSQL is a powerful database where data is organized in tables with rows and columns.

```
Think of it like:
- Excel spreadsheet on steroids
- With relationships between tables
- Built for reliability and complex queries
```

When to Use:

```
✅ User accounts and authentication
✅ Financial transactions (money critical!)
✅ E-commerce orders
✅ Complex reporting
✅ Data with relationships

❌ Massive unstructured data
❌ Real-time big data
```

Example Schema:

```
-- Users Table
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    created_at TIMESTAMP
);

-- Posts Table (related to users)
CREATE TABLE posts (
```

```
    id INT PRIMARY KEY,
    user_id INT,              -- Link to user
    content TEXT,
    created_at TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- Query to get user and their posts
SELECT users.name, posts.content
FROM users
JOIN posts ON users.id = posts.user_id
WHERE users.id = 123;
```

---

3. MongoDB (NoSQL Database)

What is it? MongoDB stores data as flexible documents (like JSON) instead of rigid table rows.

```
Think of it like:
- Instead of Excel rows: JSON documents
- Flexible schema (add fields whenever)
- Better for unstructured data
```

When to Use:

```
✅ Social media posts (flexible format)
✅ User profiles (different fields per user)
✅ IoT sensor data (varies per device)
✅ Rapid prototyping (schema changes easy)

❌ Complex transactions (need SQL)
❌ Highly relational data
```

Example Document:

```
// User with flexible structure
{
  _id: ObjectId("..."),
  name: "John",
  email: "john@email.com",
  age: 30,
  hobbies: ["reading", "gaming"],
```

```
  address: {
    city: "New York",
    zip: "10001"
  },
  posts: [
    { id: 1, title: "First Post", likes: 10 },
    { id: 2, title: "Second Post", likes: 25 }
  ]
}

// Another user with different structure
{
  _id: ObjectId("..."),
  name: "Sarah",
  email: "sarah@email.com",
  social_media: ["twitter", "instagram"],
  premium: true
  // Different fields!
}
```

Query Example:

```
// Find users in New York
db.users.find({ "address.city": "New York" })

// Update user's hobby
db.users.updateOne(
  { _id: ObjectId("...") },
  { $push: { hobbies: "cooking" } }
)

// Count posts by user
db.users.aggregate([
  { $group: { _id: "$userId", count: { $sum: 1 } } }
])
```

---

4. Elasticsearch (Search Engine)

What is it? Elasticsearch is a search engine that indexes data for lightning-fast text searches.

```
Think of it like:
- Database = Library without catalog
- Elasticsearch = Library with index
```

```
   (you find books instantly by keywords)

Perfect for: Google-like search
```

How It Works:

```
Text: "I love playing basketball"

Elasticsearch breaks it into:
- love
- playing
- basketball

When you search "basketball":
- Finds all documents with "basketball"
- Returns in milliseconds
- Even if millions of documents!
```

Real Use Cases:

```
1. E-commerce search
   - Search: "blue shoes size 10"
   - Instantly get relevant products

2. Log analysis
   - Search through billions of log lines
   - Find errors in seconds

3. Full-text search
   - Like Google search
   - Search "best pizza near me"
   - Get relevant results

4. Auto-complete
   - Type "tes" → suggests "Tesla", "Test", "Texas"
```

Example:

```
// Indexing a product
POST /products/_doc
{
  "name": "Blue Running Shoes",
  "category": "shoes",
  "price": 89.99,
  "description": "Comfortable blue running shoes with good support"
```

```
}

// Search
GET /products/_search
{
  "query": {
    "match": {
      "description": "comfortable shoes"
    }
  }
}

// Returns products matching "comfortable" and/or "shoes"
// With relevance scoring
```

---

5. Apache Kafka (Message Queue)

What is it? Kafka is a system that stores messages in a queue and delivers them to multiple consumers reliably.

```
Think of it like:
- Email queue for computers
- Each service sends messages
- Other services read messages later
- Messages don't get lost
```

Real Analogy:

```
Post Office:
- You (Producer) = Person mailing letter
- Mailbox = Kafka (holds messages)
- Postal Worker = Kafka (delivers)
- Multiple Recipients (Consumers) = Services reading messages

Letter stays in post office until all readers get it!
```

Use Cases:

```
1. Notification System
   - User buys item
   - Message sent to Kafka
   - Email service reads → sends email
   - SMS service reads → sends text
```

```
   - Both get notification independently

2. Real-time Analytics
   - Events from app sent to Kafka
   - Analytics engine reads → processes
   - Dashboard service reads → updates stats

3. Decoupling Services
   - Service A sends message to Kafka
   - Service A doesn't wait for response
   - Service B reads when ready
   - Loose coupling = flexibility
```

Example:

```
# Producer (Send message)
from kafka import KafkaProducer
import json

producer = KafkaProducer(bootstrap_servers=['localhost:9092'])

message = {
    'user_id': 123,
    'action': 'purchase',
    'product_id': 456,
    'amount': 99.99
}

producer.send('purchases', json.dumps(message).encode())

# Consumer 1 (Email Service)
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'purchases',
    bootstrap_servers=['localhost:9092']
)

for message in consumer:
    purchase = json.loads(message.value)
    send_email(purchase['user_id'], f"Purchased {purchase['product_id']}")

# Consumer 2 (Loyalty Points Service)
consumer2 = KafkaConsumer('purchases')

for message in consumer2:
```

```
    purchase = json.loads(message.value)
    add_points(purchase['user_id'], purchase['amount'] * 0.1)

# Both consumers get SAME message independently!
```

Kafka vs Email:

```
Email:
- You send an email
- Recipient reads it
- Message deleted after reading

Kafka:
- Message sent to Kafka
- Multiple services can read it
- Message stays for days/weeks
- Services can read at different times
```

---

6. RabbitMQ (Message Queue Alternative)

What is it? Similar to Kafka, but simpler and better for task queues rather than streaming.

```
Kafka = Post Office with permanent history
RabbitMQ = Express delivery service
```

When to Use Each:

```
Kafka:
- Need message history
- Multiple consumers
- Real-time streaming
- Big data

RabbitMQ:
- Simple task queues
- One consumer per message
- Lightweight
- Task processing
```

Example Task Queue:

```
# RabbitMQ: Send email task
```

```
import pika

connection = pika.BlockingConnection()
channel = connection.channel()
channel.queue_declare(queue='send_email')

# Producer: Queue task
channel.basic_publish(
    exchange='',
    routing_key='send_email',
    body='{"user_id": 123, "subject": "Welcome!"}'
)

# Consumer: Process task
def callback(ch, method, properties, body):
    task = json.loads(body)
    send_email(task['user_id'], task['subject'])

channel.basic_consume(queue='send_email', on_message_callback=callback)
channel.start_consuming()
```

---

7. AWS S3 (File Storage)

What is it? Cloud storage for files (photos, videos, documents) with extreme reliability and scalability.

```
Think of it like:
- Your laptop hard drive = limited space
- AWS S3 = Infinite storage in cloud
- Always available, never loses data
```

Real Uses:

```
1. Instagram/Facebook
   - Store billions of photos
   - Serve from any location
   - Auto-replicate for safety

2. Netflix
   - Store movie files
   - Distribute to users worldwide
   - Automatic backup

3. Backups
```

```
   - Company backups
   - Never worry about disk crash
```

Example:

```
import boto3

# Connect to S3
s3 = boto3.client('s3')

# Upload file
s3.upload_file(
    Filename='my_photo.jpg',
    Bucket='my-bucket',
    Key='photos/2024/my_photo.jpg'
)

# Download file
s3.download_file(
    Bucket='my-bucket',
    Key='photos/2024/my_photo.jpg',
    Filename='downloaded.jpg'
)

# Delete file
s3.delete_object(Bucket='my-bucket', Key='photos/2024/my_photo.jpg')

# Get URL to share
url = s3.generate_presigned_url(
    'get_object',
    Params={'Bucket': 'my-bucket', 'Key': 'photos/2024/my_photo.jpg'},
    ExpiresIn=3600  # Valid for 1 hour
)
```

Reliability:

```
- Data replicated across multiple data centers
- 99.99999999% durability (11 nines!)
- Automatically handles failures
- Geographic distribution
```

---

8. Docker (Containerization)

What is it? Docker packages your application with all its dependencies into a container - like a box that works anywhere.

```
Think of it like:
- Virtual machine = Shipping whole car (heavy)
- Docker container = Shipping just the goods (light)

Why?
- Works on developer's laptop
- Works on company server
- Works in cloud
- Guaranteed same environment everywhere
```

Real Problem It Solves:

```
Developer: "It works on my machine!"
Manager: "Why doesn't it work on production?"

Reason: Different libraries, versions, OS

Docker: "Here's entire environment packaged"
```

Example:

```
# Dockerfile: Recipe for container

FROM python:3.9            # Start with Python

WORKDIR /app               # Set folder

COPY requirements.txt .    # Copy files

RUN pip install -r requirements.txt  # Install libraries

COPY . .                   # Copy code

CMD ["python", "app.py"]   # Run app


# Build container
docker build -t my-app .

# Run container
docker run -p 5000:5000 my-app
```

```
# Now your app runs in container
# Same environment everywhere!
```

Benefits:

```
- Development = Production environment
- Easy deployment
- Scalability (run 1000 containers)
- Isolation (containers don't affect each other)
```

---

9. Kubernetes (Orchestration)

What is it? Kubernetes automatically manages Docker containers, handling scaling, updates, and failures.

```
Think of it like:
- Docker = Single server running container
- Kubernetes = Manager of 1000+ containers

Kubernetes handles:
- Running 1000 containers
- If container crashes → restart automatically
- Need more capacity → spin up more containers
- Update app → rolling updates (no downtime)
- Load distribution
```

Real Analogy:

```
Restaurant Manager:
- 10 chefs (containers)
- If chef sick → hire replacement (auto-restart)
- More customers coming → hire more chefs (scale)
- New recipe for all chefs → update gradually (rolling update)
- Distribute orders to chefs → load balancing

Kubernetes does this for containers!
```

Example:

```
# Kubernetes config: Deploy application

apiVersion: apps/v1
```

```
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3              # Run 3 copies
  selector:
    app: web-app
  template:
    spec:
      containers:
      - name: app
        image: my-app:latest
        ports:
        - containerPort: 5000

        # Resource limits
        resources:
          limits:
            cpu: "1"
            memory: "512Mi"
```

What Happens:

```
1. Kubernetes starts 3 copies of your app
2. Distributes traffic to all 3
3. One crashes? Automatically restarts
4. You need more capacity? Change replicas: 3 → 5
   Kubernetes spins up 2 more automatically
5. Update app? Rolling update (restart one at a time)
   Users never see downtime
```

---

10. Git (Version Control)

What is it? Git tracks changes to code files, allowing teams to work together and maintain history of all changes.

```
Think of it like:
- Google Docs history
- But for code
- Shows who changed what and when
- Can revert to old versions
```

Basic Concepts:

```
Repository (Repo): Folder containing all code history

Commit: Snapshot of code at a point in time
        Message: "Fixed login bug"
        Date: 2024-01-15
        Author: John

Branch: Separate line of development
        main branch = production code
        feature branch = work in progress

Merge: Combine changes from one branch to another
```

Real Example:

```
# Clone repository (get code)
git clone https://github.com/user/project.git

# Create new branch for feature
git checkout -b fix-login-bug

# Make changes
# ...edit files...

# See what changed
git status
git diff

# Save changes
git add .              # Add all changes
git commit -m "Fix login bug"  # Save with message

# Upload to server
git push origin fix-login-bug

# Request merge to main
# (Create Pull Request on GitHub)

# After review and approval
git merge fix-login-bug into main
```

Collaboration:

```
Developer 1 branch: feature-payment
Developer 2 branch: feature-dashboard
```

```
Main branch: production code

Both work independently
Both merge back to main when done
Git handles combining changes
```

Frameworks & Tools

1. Express.js (Web Framework)

What is it? Express makes it easy to build web applications in JavaScript/Node.js.

```
Think of it like:
- Building a house
- Without framework: collect bricks, mix cement (tedious)
- With framework: use construction kit (fast)

Express: Provides ready-made tools for web apps
```

Real Example:

```
const express = require('express');
const app = express();

// Route: GET /users/123
app.get('/users/:id', (req, res) => {
    const userId = req.params.id;

    // Fetch user from database
    const user = getUser(userId);

    // Send response
    res.json(user);
});

// Route: POST /users (create new user)
app.post('/users', (req, res) => {
    const newUser = req.body;

    // Save to database
    saveUser(newUser);

    res.json({ message: 'User created', user: newUser });
```

```
});

// Start server
app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Common Use:

```
- Build REST APIs
- Handle HTTP requests
- Connect to database
- Send responses
```

---

2. React (Frontend Framework)

What is it? React makes building interactive user interfaces easier by breaking UI into reusable components.

```
Think of it like:
- Building with LEGO blocks
- Each block = Component
- Combine blocks = Complex UI
- Change one block = only that block updates
```

Component Example:

```
// Reusable button component
function SubmitButton({ text, onClick }) {
    return (
        <button onClick={onClick}>
            {text}
        </button>
    );
}

// User profile component
function UserProfile({ userId }) {
    const [user, setUser] = useState(null);

    // Fetch user when component loads
    useEffect(() => {
        fetchUser(userId).then(setUser);
```

```
    }, [userId]);

    return (
        <div>
            <h1>{user?.name}</h1>
            <p>Email: {user?.email}</p>
            <SubmitButton
                text="Save"
                onClick={() => saveUser(user)}
            />
        </div>
    );
}
```

Benefits:

```
- Reusable components
- Real-time updates (no page refresh)
- Efficient updates (only changed parts)
- Large ecosystem of libraries
```

---

3. FastAPI (Python Web Framework)

What is it? FastAPI makes building fast, modern APIs in Python very easy with automatic validation and documentation.

```
Think of it like:
- Flask = Manual construction
- FastAPI = Pre-built, automatic

FastAPI:
- Automatic API validation
- Built-in documentation
- Type hints support
- Very fast performance
```

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
```

```
# Data model
class User(BaseModel):
    id: int
    name: str
    email: str

# Database (mock)
users_db = {}

# GET endpoint
@app.get("/users/{user_id}")
async def get_user(user_id: int):
    """Get user by ID"""
    return users_db.get(user_id)

# POST endpoint
@app.post("/users")
async def create_user(user: User):
    """Create new user"""
    users_db[user.id] = user
    return {"message": "User created", "user": user}

# PUT endpoint
@app.put("/users/{user_id}")
async def update_user(user_id: int, user: User):
    """Update user"""
    users_db[user_id] = user
    return user

# DELETE endpoint
@app.delete("/users/{user_id}")
async def delete_user(user_id: int):
    """Delete user"""
    del users_db[user_id]
    return {"message": "User deleted"}

# Automatic validation
# Try: POST /users with invalid data → FastAPI rejects with error
# Try: GET /docs → Automatic interactive documentation!
```

Special Features:

```
- Type hints: user: User → auto validation
- OpenAPI docs: /docs shows all endpoints automatically
- Async support: Handle many requests simultaneously
- Very fast: One of fastest Python frameworks
```

4. Django (Python Web Framework)

What is it? Django is a complete web framework for Python with built-in admin panel, ORM, and security features.

```
Think of it like:
- FastAPI = Minimalist, fast
- Django = Full-featured, batteries included

Django includes:
- Database ORM
- Admin interface
- Authentication
- Migrations
- Form validation
```

Real Example:

```
# models.py (Database schema)
from django.db import models

class User(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'users'

class Post(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

# views.py (Handle requests)
from django.shortcuts import render
from django.http import JsonResponse

def get_user(request, user_id):
    user = User.objects.get(id=user_id)
    posts = Post.objects.filter(user=user)

    return JsonResponse({
        'user': user.name,
```

```
        'posts': [p.title for p in posts]
    })

# urls.py (Map URLs to views)
from django.urls import path
from . import views

urlpatterns = [
    path('users/<int:user_id>/', views.get_user),
]

# admin.py (Auto admin interface!)
from django.contrib import admin

admin.site.register(User)
admin.site.register(Post)

# Go to /admin → Login → Manage all data with UI!
```

Common Use:

```
✅ Complete web applications
✅ Rapid development
✅ Admin interface needed
✅ Complex database operations

❌ Simple APIs (FastAPI better)
❌ Real-time applications
```

---

5. Nginx (Web Server & Reverse Proxy)

What is it? Nginx is a high-performance web server that serves web pages and forwards requests to backend servers.

```
Think of it like:
- Restaurant manager at front door
- You (client) make request
- Nginx directs you to correct server
- Or serves cached response immediately
```

Main Uses:

1. Web Server (Serve static files)

```
server {
    listen 80;
    server_name example.com;

    # Serve static files
    location / {
        root /var/www/html;
        index index.html;
    }
}
```

2. Reverse Proxy (Forward to backend servers)

```
upstream backend {
    server backend1.example.com;
    server backend2.example.com;
    server backend3.example.com;
}

server {
    listen 80;

    location / {
        proxy_pass http://backend;  # Forward to backend servers
        proxy_set_header Host $host;
    }
}
```

3. Load Balancer (Distribute traffic)

```
upstream app_servers {
    server 192.168.1.1:8000 weight=3;  # Gets more traffic
    server 192.168.1.2:8000;
    server 192.168.1.3:8000;
}

server {
    location / {
        proxy_pass http://app_servers;  # Round-robin distribution
    }
}
```

4. Caching (Speed up responses)

```
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache:10m;
```

```
server {
    location / {
        proxy_cache my_cache;
        proxy_cache_valid 200 10m;  # Cache for 10 minutes
        proxy_pass http://backend;
    }
}
```

---

6. Docker Compose (Multi-container Management)

What is it? Docker Compose runs multiple containers together easily with a simple configuration file.

```
Think of it like:
- Docker = Run one container
- Docker Compose = Run 5 containers together

Example:
- Backend container
- Database container
- Redis container
- Nginx container
- All talking to each other
```

Example:

```
# docker-compose.yml

version: '3'
services:

  # Backend service
  backend:
    build: .
    ports:
      - "8000:8000"
    environment:
      DB_HOST: postgres
      REDIS_HOST: redis
    depends_on:
      - postgres
      - redis
```

```
  # Database service
  postgres:
    image: postgres:13
    environment:
      POSTGRES_DB: myapp
      POSTGRES_PASSWORD: secret
    volumes:
      - postgres_data:/var/lib/postgresql/data

  # Cache service
  redis:
    image: redis:6
    ports:
      - "6379:6379"

  # Web server service
  nginx:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - backend

volumes:
  postgres_data:


# Run everything with one command!
docker-compose up

# All 4 services start
# Connected to each other
# Database persisted
# Logs shown together

# Stop everything
docker-compose down
```

---

7. Prometheus (Metrics Collection)

What is it? Prometheus collects metrics (numbers) from your application to monitor health and performance.

```
Think of it like:
- Your app = Patient
- Prometheus = Doctor measuring vitals
- Metrics: Heart rate, temperature, blood pressure
```

Metrics Example:

```
from prometheus_client import Counter, Histogram, Gauge

# Counter: Increases only
request_count = Counter('http_requests_total', 'Total requests')

# Gauge: Can go up or down
active_users = Gauge('active_users', 'Currently active users')

# Histogram: Distribution of values
response_time = Histogram('response_time_seconds', 'Response time')

# In your code
request_count.inc()                    # Count request
active_users.set(100)                  # Set current users
response_time.observe(0.25)            # Record time
```

What Gets Monitored:

```
- CPU usage
- Memory usage
- Requests per second
- Error rates
- Database query time
- Cache hit rate
- Everything!
```

---

8. Grafana (Visualization)

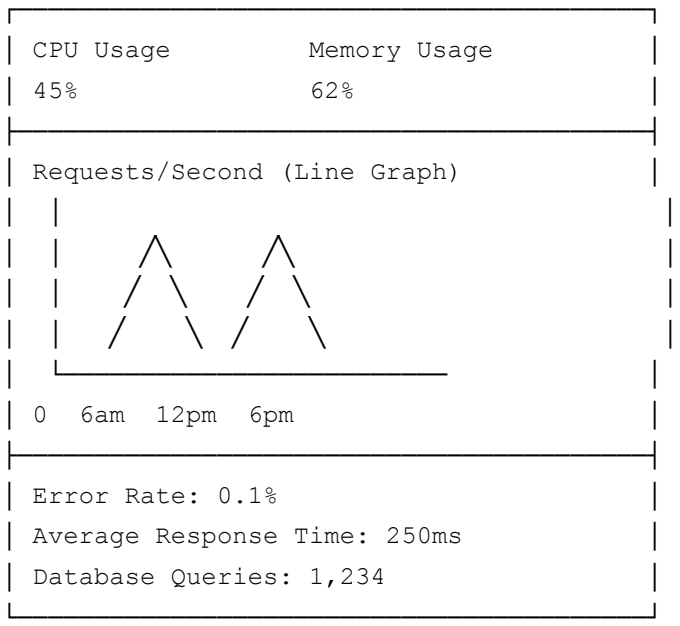
What is it? Grafana creates beautiful dashboards from metric data collected by Prometheus.

```
Think of it like:
- Prometheus = Data collector
- Grafana = Dashboard creator

Prometheus: "Response time is 0.25 seconds"
```

```
Grafana: Shows beautiful line graph of response times
```

Dashboard Example:



Setup:

```
# In docker-compose.yml
prometheus:
  image: prom/prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml

grafana:
  image: grafana/grafana
  ports:
    - "3000:3000"
  depends_on:
    - prometheus
```

PART 2: ADVANCED TOPICS & SYSTEM DESIGN

Scalability Fundamentals

Key Concepts

Vertical Scaling (Scale-up)

- Add more power to a single machine (CPU, RAM, Storage)
- Easier to implement initially
- Limited by single machine capacity
- Usually cheaper at first, expensive at scale
- Single point of failure risk

Horizontal Scaling (Scale-out)

- Add more machines to the system
- No single point of failure
- More complex coordination needed
- Better cost-efficiency at large scale
- Can scale indefinitely (theoretically)

Scalability Metrics

```
Throughput = Requests processed per second (RPS/QPS)
Latency = Time to process one request (milliseconds)
Availability = % of time system is operational (99.9%, 99.99%, etc.)
```

Back of Envelope Calculation Template

```
Monthly Active Users (MAU): 100M
Daily Active Users (DAU): 10M (10% of MAU)
QPS (avg): DAU * Avg Requests per User / Seconds in a Day
         = 10M * 100 / (24 * 3600)
         = ~11,600 QPS

Peak QPS: 3-5x average = 35,000-58,000 QPS

Storage:
Per user data: 1KB
Total: 100M * 1KB = 100GB
Per year growth: 100GB * 365 = 36.5TB
```

Load Balancing

Question 1: Design a Load Balancer

Problem Statement

You need to distribute incoming traffic across multiple servers to:

- Prevent any single server from being overwhelmed
- Maximize throughput and minimize response time
- Detect failed servers and route around them
- Support millions of concurrent connections

Requirements

Functional:

- Distribute requests to backend servers
- Health checks on servers
- Session persistence (sticky sessions when needed)
- Support multiple load balancing algorithms

Non-Functional:

- Sub-millisecond latency overhead
- 99.99% availability
- Handle 100K+ concurrent connections

High-Level Architecture

```
graph TB
    Client["👥 Clients"]
    LB1["Load Balancer 1"]
    LB2["Load Balancer 2 (Backup)"]
    Server1["Web Server 1"]
    Server2["Web Server 2"]
    Server3["Web Server 3"]
    HealthCheck["Health Check Service"]
```

```
Client -->|Request| LB1
Client -->|Failover| LB2
LB1 -->|Route| Server1
LB1 -->|Route| Server2
LB1 -->|Route| Server3
LB2 -->|Route| Server1
LB2 -->|Route| Server2
LB2 -->|Route| Server3
HealthCheck -->|Periodic Check| Server1
HealthCheck -->|Periodic Check| Server2
HealthCheck -->|Periodic Check| Server3
```

Load Balancing Algorithms

Algorithm	Best For	Trade-offs
Round Robin	Equal capacity servers	Doesn't account for load variations
Least Connections	Long-lived connections	Overhead of tracking connections
IP Hash	Sticky sessions	Unequal distribution if servers added/removed
Weighted Round Robin	Different server capacities	Requires manual configuration
Consistent Hashing	Distributed caching, minimal rehashing	More complex implementation

Real-World Example: Netflix

Netflix uses **AWS ELB (Elastic Load Balancer)** with:

- Application Load Balancer (ALB) for HTTP/HTTPS
- Network Load Balancer (NLB) for extreme performance
- Geographic load balancing across regions
- Health checks every 5 seconds
- Automatic failover in milliseconds

Implementation: Least Connections Algorithm

```
class LoadBalancer:
    def __init__(self, servers):
        self.servers = servers
        self.connections = {server: 0 for server in servers}

    def get_server(self):
        """Route to server with least connections"""
        return min(self.servers, key=lambda s: self.connections[s])

    def route_request(self, request):
        server = self.get_server()
        self.connections[server] += 1

        try:
            response = server.handle(request)
            return response
        finally:
            self.connections[server] -= 1

    def health_check(self, server):
        """Periodic health check"""
        try:
            server.ping()
            return True
        except:
            self.servers.remove(server)
            return False
```

Scenario-Based Follow-ups

Q1: What if a load balancer itself fails?

- A: Use redundant load balancers with failover mechanism
- Secondary LB ready to take over immediately
- Use DNS failover or Virtual IP (VIP) with VRRP
- Multiple LBs distributed across availability zones

Q2: How do you handle sticky sessions?

- A: Use IP hash or session tokens
- Store session data in distributed cache (Redis)
- Cookie-based routing with consistent hashing

- Trade-off: Reduces load distribution efficiency

Q3: What if you have 1M concurrent connections?

- A: Layer load balancing (DNS → LB → Server LB)
- Use specialized hardware load balancers (F5, Citrix)
- Connection pooling on servers
- Optimize TCP settings (backlog, timeout)

Interview Tips

✅ **Do:**

- Start with simple algorithms (Round Robin), then optimize
- Discuss health check frequency vs overhead
- Mention geographic distribution
- Consider session affinity requirements

❌ **Avoid:**

- Single point of failure in LB design
- Ignoring network latency
- Assuming equal server capacity
- Forgetting about graceful degradation

Caching Strategies

Question 2: Design a Distributed Caching System

Problem Statement

Design a system to cache frequently accessed data across multiple servers to reduce database load and improve response times. Handle:

- Millions of cache requests per second
- Data consistency across nodes

- Eviction policies when cache is full
- Cache failures and recovery

Requirements

Functional:

- Get/Set/Delete operations
- Expiration (TTL)
- Multiple eviction policies
- Replication for redundancy

Non-Functional:

- Sub-millisecond latency
- 99.99% availability
- Support 1M+ QPS
- Memory efficient

Caching Strategies

```
graph LR
    Client["Client"]
    AppServer["Application Server"]
    Cache["Cache Layer<br/>(Redis/Memcached)"]
    DB["Database"]

    Client -->|Request| AppServer
    AppServer -->|Check Cache| Cache

    Cache -->|Hit| AppServer
    Cache -->|Miss| DB
    DB -->|Fetch Data| AppServer
    AppServer -->|Store in Cache| Cache
    AppServer -->|Response| Client
```

Cache Replacement Policies

Policy	Description	Use Case

LRU (Least Recently Used)	Remove least recently accessed item	General purpose, working sets
LFU (Least Frequently Used)	Remove least frequently accessed	Long-tail content
FIFO	Remove oldest entry	Simple, predictable
Random	Remove random entry	Quick, uniform distribution

Caching Patterns

1. Cache-Aside (Lazy Loading)

```
Application:
- Check cache for data
- If miss, fetch from DB
- Update cache
- Return to user

Pros: Simple, data freshness
Cons: Cache miss penalty
```

2. Write-Through

```
Application:
- Write to cache AND database simultaneously
- Both operations succeed or both fail

Pros: Data consistency
Cons: Slower writes, cache always fresh
```

3. Write-Behind (Write-Back)

```
Application:
- Write to cache immediately
- Asynchronously write to database

Pros: Fast writes
Cons: Data loss risk if cache fails
```

4. Refresh-Ahead

```
System:
```

```
- Proactively refresh data before expiration
- Update TTL based on access patterns

Pros: No cache misses
Cons: Unnecessary refreshes if not accessed
```

Real-World Example: Instagram

Instagram's caching strategy:

- **Redis** for session data, feed cache, and user info (99% hit rate)
- **Memcached** for expensive computations
- Multi-layer caching:
  - CDN layer (edge caching)
  - Application layer (Redis)
  - Database layer (query result caching)
- Cache invalidation on user actions (post, like, follow)
- Consistent hashing for distributed cache
- Replica nodes for failover

Implementation: Distributed Cache

```
import hashlib
from collections import OrderedDict
from datetime import datetime, timedelta

class DistributedCache:
    def __init__(self, max_size=1000, ttl=3600):
        self.cache = OrderedDict()
        self.max_size = max_size
        self.ttl = ttl
        self.access_count = {}

    def _hash_key(self, key):
        """Hash key for consistent hashing"""
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def get(self, key):
        """Get value from cache"""
```

```
        if key not in self.cache:
            return None

        value, expiry = self.cache[key]

        # Check expiration
        if datetime.now() > expiry:
            del self.cache[key]
            return None

        # Update LRU
        self.cache.move_to_end(key)
        self.access_count[key] = self.access_count.get(key, 0) + 1
        return value

    def set(self, key, value):
        """Set value in cache"""
        if len(self.cache) >= self.max_size:
            # Evict least recently used
            evicted_key, _ = self.cache.popitem(last=False)
            del self.access_count[evicted_key]

        self.cache[key] = (value, datetime.now() + timedelta(seconds=self.ttl))
        self.cache.move_to_end(key)
        self.access_count[key] = 1

    def delete(self, key):
        """Delete value from cache"""
        if key in self.cache:
            del self.cache[key]
            del self.access_count[key]
```

Scenario-Based Follow-ups

Q1: How do you handle cache stampede?

- A: Multiple requests hit cache miss simultaneously, all hit database
- Solution: Probabilistic early expiration (XFetch)
- Or: Use lock/mutex for cache population
- Or: Preload data before expiration

Q2: How do you maintain cache consistency?

- A: Write-through for consistency

- Or: Event-driven invalidation
- Or: Versioning/tagging strategy
- Or: Time-based expiration with occasional refresh

Q3: What happens when cache node dies?

- A: Consistent hashing minimizes rebalancing
- Replication nodes provide backup
- Rebuild from database (with rate limiting)
- Use cache warming for critical data

Interview Tips

✅ **Do:**

- Understand cache-aside vs write-through patterns
- Discuss TTL and expiration strategies
- Mention consistent hashing for scaling
- Cover failure scenarios and recovery

❌ **Avoid:**

- Treating cache as persistent storage
- Ignoring cache invalidation
- Assuming all data should be cached
- Missing replication strategy

Database Design

Question 3: SQL vs NoSQL - When to Use Each?

Comparison Table

Aspect	SQL (Relational)	NoSQL (Non-Relational)
Schema	Fixed, predefined	Flexible, dynamic

ACID	Full ACID support	Eventually consistent (BASE)
Scalability	Vertical (sharding complex)	Horizontal (built-in)
Queries	Complex joins	Simple, document-based
Transactions	Multi-row ACID	Single-document ACID
Use Cases	Financial, relational data	Big data, real-time, unstructured

SQL Decision Tree

```
graph TD
    A["Start: Choose Database Type"] --> B{"Do you need<br/>multi-row<br/>transac
    B -->|Yes| C["Use SQL"]
    B -->|No| D{"Highly<br/>relational<br/>data?"}
    D -->|Yes| C
    D -->|No| E{"Flexible<br/>schema<br/>needed?"}
    E -->|Yes| F["Use NoSQL"]
    E -->|No| C
    C --> G["PostgreSQL, MySQL<br/>Oracle, SQL Server"]
    F --> H["MongoDB, Cassandra<br/>DynamoDB, Firestore"]
```

Real-World Examples

SQL Preferred:

- **Twitter**: User accounts, tweets (immutable), relationships
- **Stripe**: Payment transactions (ACID critical)
- **Healthcare Systems**: Patient records (compliance, relationships)

NoSQL Preferred:

- **Netflix**: User profiles, viewing history (flexible schema)
- **Uber**: Real-time location data (high write throughput)
- **Spotify**: Music metadata (massive scale, simple queries)

Question 4: Database Sharding Strategy

Problem Statement

A single database can’t handle the scale:

- 100TB+ of data
- Millions of writes per second
- Need sub-second latency

Design a sharding strategy that allows horizontal scaling while maintaining data consistency.

Sharding Approaches

```
graph LR
    A["Client Request<br/>UserID: 12345"]
    B["Hash Function<br/>12345 % 4 = 1"]
    C["Route to Shard"]
    D["Shard 0<br/>UserID % 4 = 0"]
    E["Shard 1<br/>UserID % 4 = 1"]
    F["Shard 2<br/>UserID % 4 = 2"]
    G["Shard 3<br/>UserID % 4 = 3"]

    A --> B --> C
    C -->|Shard 1| E
    C -.->|Unused| D
    C -.->|Unused| F
    C -.->|Unused| G
```

Strategy	Method	Pros	Cons
Range	Shard by value range	Simple	Uneven distribution, hotspots
Hash	Hash on key mod N	Even distribution	Rebalancing hard when adding shards
Consistent Hash	Hash-based with virtual nodes	Even distribution, easy rebalancing	More complex
Directory	Lookup table of shard mappings	Flexible	Single point of failure, lookup overhead

Sharding Key Selection

Good Sharding Keys:

- User ID (for user-centric systems like Twitter)
- Customer ID (for e-commerce)
- Tenant ID (for multi-tenant SaaS)
- Geographic region (for geo-distributed data)

Bad Sharding Keys:

- Timestamps (create hotspots as data streams in)
- Email (many users from same domain)
- Boolean flags (create two huge shards)

Database Replication Architecture

```
graph TB
    Client["Client"]
    LB["Load Balancer"]
    Primary["Primary DB<br/>(Write)"]
    Replica1["Replica 1<br/>(Read)"]
    Replica2["Replica 2<br/>(Read)"]
    Replica3["Replica 3<br/>(Read)"]

    Client -->|Write| LB
    Client -->|Read| LB
    LB -->|Write| Primary
    LB -->|Read| Replica1
    LB -->|Read| Replica2
    LB -->|Read| Replica3

    Primary -->|Binlog Stream| Replica1
    Primary -->|Binlog Stream| Replica2
    Primary -->|Binlog Stream| Replica3
```

Implementation: Consistent Hashing

```
import hashlib

class ConsistentHashRing:
    def __init__(self, nodes=None, virtual_nodes=150):
        self.virtual_nodes = virtual_nodes
        self.ring = {}
        self.sorted_keys = []
```

```
        if nodes:
            for node in nodes:
                self.add_node(node)

    def _hash(self, key):
        """Hash function"""
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node):
        """Add node with virtual replicas"""
        for i in range(self.virtual_nodes):
            virtual_key = f"{node}:{i}"
            hash_value = self._hash(virtual_key)
            self.ring[hash_value] = node

        self.sorted_keys = sorted(self.ring.keys())

    def remove_node(self, node):
        """Remove node and its virtual replicas"""
        for i in range(self.virtual_nodes):
            virtual_key = f"{node}:{i}"
            hash_value = self._hash(virtual_key)
            del self.ring[hash_value]

        self.sorted_keys = sorted(self.ring.keys())

    def get_node(self, key):
        """Get node for key"""
        if not self.ring:
            return None

        hash_value = self._hash(key)

        # Find first node clockwise
        for ring_key in self.sorted_keys:
            if ring_key >= hash_value:
                return self.ring[ring_key]

        # Wrap around
        return self.ring[self.sorted_keys[0]]

# Usage
ring = ConsistentHashRing(['db-shard-1', 'db-shard-2', 'db-shard-3'])
user_id = 'user:12345'
shard = ring.get_node(user_id)  # Get which shard for user
```

Real-World Example: Pinterest

Pinterest's database architecture:

- **Sharding strategy**: User ID based sharding
- **900+ shards** across multiple clusters
- Replication: 3 replicas per shard
- Read-write split: Writes to primary, reads distributed
- **Shard migration**: Gradual background process
- **Hot shard handling**: Breaking large shards, adding capacity

Scenario-Based Follow-ups

Q1: User 12345 becomes very active (hotspot). How to handle?

- A: Shard splitting strategy
- Create new shards: `shard_x` and `shard_y` from `shard_x`
- Use consistent hashing with gradual migration
- Or: Add caching layer for hot data
- Or: Use write sharding + read replication

Q2: How do you handle cross-shard transactions?

- A: Generally avoid if possible (design for single shard)
- Use 2-phase commit (complex, slow)
- Use sagas/compensating transactions
- Or: Denormalize to avoid joins

Q3: How do you expand from 4 to 8 shards?

- A: Consistent hashing minimizes movement (~50% of keys)
- Background migration job transfers data
- Dual-write period during transition
- Validation and rollback capability

Interview Tips

✅ **Do:**

- Choose sharding key based on access patterns
- Discuss read replicas vs shards
- Mention replication lag and consistency
- Cover failover and recovery

❌ **Avoid:**

- Sharding on mutable attributes
- Ignoring uneven distribution
- Assuming no cross-shard queries needed
- Missing replication strategy

Message Queues & Event-Driven Architecture

Question 5: Design a Notification System

Problem Statement

Design a system to send notifications (email, SMS, push) to millions of users:

- Handle millions of messages per second
- Guarantee delivery with retries
- Support multiple channels
- Handle rate limiting per user

Requirements

Functional:

- Queue notifications
- Multi-channel delivery (email, SMS, push)
- Retry failed sends
- Track delivery status

Non-Functional:

- Sub-second latency for queueing
- 99.99% delivery rate
- Horizontal scaling

Architecture

```
graph TB
    API["Notification API"]
    Queue["Message Queue<br/>(RabbitMQ/Kafka)"]
    Consumer["Notification Worker"]
    EmailService["Email Service<br/>(SendGrid)"]
    SMSService["SMS Service<br/>(Twilio)"]
    PushService["Push Service<br/>(FCM)"]
    DB[(Database)]

    API -->|Push Message| Queue
    Queue -->|Consume| Consumer
    Consumer -->|Route by Type| EmailService
    Consumer -->|Route by Type| SMSService
    Consumer -->|Route by Type| PushService
    Consumer -->|Update Status| DB
    EmailService -->|Status| DB
    SMSService -->|Status| DB
    PushService -->|Status| DB
```

Message Queue Comparison

Feature	RabbitMQ	Kafka	AWS SQS
Throughput	50K msg/s	1M msg/s	300K msg/s
Persistence	Optional	Permanent log	Automatic
Ordering	Per queue	Per partition	FIFO queues
Replay	No	Yes (log-based)	Limited
Use Case	Task queues	Event streaming	Simple queueing

Notification Service Implementation

```
import json
import time
from enum import Enum
from dataclasses import dataclass
from typing import Optional

class NotificationType(Enum):
    EMAIL = "email"
    SMS = "sms"
    PUSH = "push"

class DeliveryStatus(Enum):
    PENDING = "pending"
    SENT = "sent"
    FAILED = "failed"
    BOUNCED = "bounced"

@dataclass
class Notification:
    user_id: str
    type: NotificationType
    content: str
    recipient: str  # email, phone, or device_id
    priority: int = 1  # 1 (high) to 5 (low)
    retry_count: int = 0
    max_retries: int = 3

class NotificationQueue:
    def __init__(self, queue_name, connection):
        self.queue_name = queue_name
        self.connection = connection  # RabbitMQ/Kafka connection

    def enqueue(self, notification: Notification):
        """Add notification to queue"""
        message = {
            'user_id': notification.user_id,
            'type': notification.type.value,
            'content': notification.content,
            'recipient': notification.recipient,
            'priority': notification.priority,
            'retry_count': notification.retry_count
        }
        self.connection.publish(
            exchange='notifications',
            routing_key=notification.type.value,
            body=json.dumps(message),
            priority=notification.priority
```

```
        )

    def consume(self, callback):
        """Consume and process messages"""
        while True:
            message = self.connection.consume(queue=self.queue_name)
            if message:
                try:
                    notification = self._parse_message(message)
                    callback(notification)
                except Exception as e:
                    self._handle_error(notification, e)

class NotificationWorker:
    def __init__(self, queue: NotificationQueue):
        self.queue = queue

    def process_notification(self, notification: Notification):
        """Route notification to appropriate service"""
        try:
            if notification.type == NotificationType.EMAIL:
                self._send_email(notification)
            elif notification.type == NotificationType.SMS:
                self._send_sms(notification)
            elif notification.type == NotificationType.PUSH:
                self._send_push(notification)

            self._update_status(notification, DeliveryStatus.SENT)
        except Exception as e:
            self._handle_failure(notification, e)

    def _handle_failure(self, notification: Notification, error: Exception):
        """Retry failed notifications"""
        notification.retry_count += 1

        if notification.retry_count < notification.max_retries:
            # Exponential backoff
            delay = 2 ** notification.retry_count
            self.queue.enqueue_with_delay(notification, delay)
        else:
            self._update_status(notification, DeliveryStatus.FAILED)

    def _send_email(self, notification: Notification):
        # Integration with SendGrid/SES
        pass

    def _send_sms(self, notification: Notification):
```

```
        # Integration with Twilio
        pass

    def _send_push(self, notification: Notification):
        # Integration with Firebase Cloud Messaging
        pass
```

Real-World Example: Uber

Uber's notification system:

- **Kafka** for event streaming
- Multi-channel delivery: Push, SMS, Email
- Per-user rate limiting: 100 notifications/hour
- Delivery status tracking in real-time
- A/B testing for notification timing
- Retry with exponential backoff (1s, 2s, 4s, 8s)
- 99.5% delivery rate

Scenario-Based Follow-ups

Q1: How do you handle notification storms?

- A: Rate limiting per user/channel
- Priority queue for critical notifications
- Batch processing during off-peak hours
- Load shedding for non-critical messages

Q2: How to ensure delivery?

- A: Persistent queue (RabbitMQ, Kafka)
- Multiple replicas of queue
- Delivery confirmation from service
- Dead-letter queue for undeliverable messages

Q3: User gets duplicate notifications. How to fix?

- A: Idempotency keys (deduplication)
- Database uniqueness constraint
- Consumer deduplication window (24 hours)

Interview Tips

✅ **Do:**

- Design for idempotency
- Discuss retry strategies
- Mention rate limiting
- Cover failure scenarios

❌ **Avoid:**

- Synchronous delivery
- Ignoring duplicate messages
- Missing retry logic
- Assuming instant delivery

API Design & REST Principles

Question 6: Design RESTful API for E-Commerce Platform

REST Principles (HATEOAS)

```
H - Hypermedia: Include links to related resources
A - As
T - The
E - Engine
O - Of
A - Application
S - State
```

HTTP Methods & Status Codes

Method	Purpose	Idempotent
GET	Retrieve resource	✅ Yes
POST	Create resource	❌ No
PUT	Replace resource	✅ Yes
PATCH	Partial update	❌ No
DELETE	Remove resource	✅ Yes

Status Code Categories

```
2xx - Success
  200: OK
  201: Created
  204: No Content

3xx - Redirection
  301: Moved Permanently
  304: Not Modified

4xx - Client Error
  400: Bad Request
  401: Unauthorized
  403: Forbidden
  404: Not Found
  429: Too Many Requests

5xx - Server Error
  500: Internal Server Error
  503: Service Unavailable
```

E-Commerce API Design

```
GET /products
GET /products/{product_id}
POST /products
PUT /products/{product_id}
DELETE /products/{product_id}

GET /users/{user_id}/cart
POST /users/{user_id}/cart/items
DELETE /users/{user_id}/cart/items/{item_id}
```

```
POST /orders
GET /orders/{order_id}
PATCH /orders/{order_id}/status

POST /payments
GET /payments/{payment_id}
```

API Response Format

```
{
  "status": "success",
  "code": 200,
  "data": {
    "product_id": "prod_123",
    "name": "Laptop",
    "price": 999.99,
    "stock": 50,
    "links": {
      "self": "/products/prod_123",
      "collection": "/products",
      "reviews": "/products/prod_123/reviews"
    }
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00Z",
    "version": "1.0"
  }
}
```

Error Response Format

```
{
  "status": "error",
  "code": 400,
  "error": {
    "type": "VALIDATION_ERROR",
    "message": "Invalid product quantity",
    "details": [
      {
        "field": "quantity",
        "message": "Must be between 1 and 100"
      }
    ]
```

```
  },
  "meta": {
    "timestamp": "2024-01-15T10:30:00Z",
    "request_id": "req_xyz_789"
  }
}
```

API Rate Limiting Headers

```
X-RateLimit-Limit: 1000 (requests per hour)
X-RateLimit-Remaining: 456 (requests left)
X-RateLimit-Reset: 1234567890 (Unix timestamp)

Retry-After: 60 (seconds to wait before retry)
```

Implementation: Rate Limiting with Sliding Window

```
from collections import deque
from time import time
from threading import Lock

class RateLimiter:
    def __init__(self, max_requests, window_seconds):
        self.max_requests = max_requests
        self.window = window_seconds
        self.requests = {}
        self.lock = Lock()

    def is_allowed(self, user_id):
        """Check if request is allowed"""
        with self.lock:
            now = time()

            if user_id not in self.requests:
                self.requests[user_id] = deque()

            # Remove old requests outside window
            while self.requests[user_id] and \
                  self.requests[user_id][0] < now - self.window:
                self.requests[user_id].popleft()

            # Check limit
            if len(self.requests[user_id]) < self.max_requests:
                self.requests[user_id].append(now)
```

```
                return True

            return False

# Usage
limiter = RateLimiter(max_requests=100, window_seconds=3600)

if limiter.is_allowed("user_123"):
    # Process request
    pass
else:
    # Return 429 Too Many Requests
    pass
```

Real-World Example: Stripe API

Stripe's API design:

- RESTful endpoints with semantic versioning
- Request ID for traceability
- Webhook events for asynchronous notifications
- Rate limiting: 25 requests/second
- Error codes for client/server/rate-limit issues
- Idempotent keys for safe retries

Interview Tips

✅ **Do:**

- Use appropriate HTTP methods
- Include pagination for large results
- Design for versioning (v1, v2)
- Document error scenarios

❌ **Avoid:**

- Using POST for GET operations
- Inconsistent naming conventions
- Missing input validation

- Ignoring idempotency

Microservices Architecture

Question 7: Design Uber with Microservices

Microservices Decomposition

Service Communication Patterns

Pattern	Synchronous	Asynchronous
REST/HTTP	✅	❌
gRPC	✅	❌
Message Queue	❌	✅
Event Bus	❌	✅
Pub/Sub	❌	✅

API Gateway Pattern

```
graph LR
    Client["Client"]
    Gateway["API Gateway"]
    US["User Service"]
    RS["Ride Service"]
    DS["Driver Service"]

    Client -->|/user/*| Gateway
    Client -->|/ride/*| Gateway
    Client -->|/driver/*| Gateway

    Gateway -->|Route| US
    Gateway -->|Route| RS
    Gateway -->|Route| DS
```

API Gateway Responsibilities

```
1. Request Routing: Route to correct service
2. Load Balancing: Distribute across instances
3. Rate Limiting: Throttle per user/client
4. Authentication: Validate JWT tokens
5. Protocol Translation: HTTP to gRPC
6. Caching: Cache GET responses
7. Request/Response Logging: For auditing
8. API Versioning: Support multiple versions
```

Distributed Transaction: Saga Pattern

```
sequenceDiagram
```

```
    participant Client
    participant RideService as Ride Service
    participant PaymentService as Payment Service
    participant DriverService as Driver Service

    Client->>RideService: Create Ride
    RideService->>RideService: Reserve Ride
    RideService-->>Client: Ride ID

    Client->>PaymentService: Process Payment
    PaymentService->>PaymentService: Charge Card
    alt Payment Success
        PaymentService-->>Client: Success
        PaymentService->>DriverService: Notify Driver
    else Payment Failed
        PaymentService-->>Client: Failed
        PaymentService->>RideService: Compensate (Cancel)
        RideService->>RideService: Release Ride
    end
```

Real-World Example: Amazon

Amazon's microservices:

- 150,000+ microservices
- Each team owns 2-3 services
- Independent deployment and scaling
- Service mesh (using internally developed tools)
- Eventual consistency model
- Internal event bus for service communication

Interview Tips

✅ **Do:**

- Start with monolith, then decompose
- Define clear service boundaries
- Discuss communication patterns
- Handle distributed transactions

❌ **Avoid:**

- Too many fine-grained services
- Synchronous calls everywhere
- Ignoring eventual consistency
- Missing circuit breakers

Real-Time Systems

Question 8: Design Instagram Feed with Real-Time Updates

Problem Statement

Design a system that:

- Delivers feed updates in real-time (< 1 second latency)
- Handles millions of concurrent WebSocket connections
- Supports efficient fan-out to followers

WebSocket Architecture

```
graph TB
    Client["Mobile Client<br/>WebSocket"]
    LB["Load Balancer"]
    WSServer1["WebSocket Server 1"]
    WSServer2["WebSocket Server 2"]
    Redis["Redis<br/>Pub/Sub"]
    PostService["Post Service"]
    FeedCache["Feed Cache<br/>Redis"]

    Client -->|WebSocket| LB
    LB --> WSServer1
    LB --> WSServer2

    PostService -->|Publish Event| Redis
    Redis -->|Subscribe| WSServer1
    Redis -->|Subscribe| WSServer2

    WSServer1 -->|Push Update| Client
    WSServer2 -->|Push Update| Client
```

Feed Generation Strategy

Push Model (Real-time):

```
When user posts:
- For each follower:
  - Add post to feed cache
  - Send notification via WebSocket

Pros: Real-time updates
Cons: High write amplification (1 write → N followers)
```

Pull Model (On-demand):

```
When user opens app:
- Fetch posts from followed accounts
- Merge and rank in real-time

Pros: No fan-out overhead
Cons: High read latency
```

Hybrid Model (Best):

```
Push for active users
Pull for inactive users
Balance: Push for friends, Pull for distant users
```

WebSocket Connection Management

```
import asyncio
import json
from collections import defaultdict
from typing import Set

class WebSocketManager:
    def __init__(self, redis_client):
        self.redis = redis_client
        self.connections: dict[str, Set] = defaultdict(set)

    async def connect(self, user_id: str, websocket):
        """Register user connection"""
        self.connections[user_id].add(websocket)
```

```
        # Subscribe to user's feed updates
        await self.redis.subscribe(f"feed:{user_id}")

    async def disconnect(self, user_id: str, websocket):
        """Remove user connection"""
        self.connections[user_id].discard(websocket)

        if not self.connections[user_id]:
            await self.redis.unsubscribe(f"feed:{user_id}")

    async def broadcast_to_user(self, user_id: str, message: dict):
        """Send message to all connections of user"""
        for websocket in self.connections[user_id]:
            try:
                await websocket.send_json(message)
            except Exception as e:
                # Connection closed
                self.connections[user_id].discard(websocket)

    async def handle_post(self, post_data: dict):
        """When user posts, notify all followers"""
        user_id = post_data['user_id']
        followers = await self.get_followers(user_id)

        # Push to followers' feeds
        for follower_id in followers:
            message = {
                'type': 'new_post',
                'data': post_data
            }
            await self.broadcast_to_user(follower_id, message)
```

Pub/Sub Pattern with Redis

```
import asyncio
import redis.asyncio as redis

class FeedPubSub:
    def __init__(self, redis_url):
        self.redis_url = redis_url

    async def publish_post(self, post_id, user_id):
        """Publish new post to followers"""
        r = await redis.from_url(self.redis_url)
```

```
        followers = await self.get_followers(user_id)

        for follower_id in followers:
            channel = f"feed:{follower_id}"
            message = {
                'post_id': post_id,
                'user_id': user_id,
                'timestamp': time.time()
            }
            await r.publish(channel, json.dumps(message))

    async def subscribe_to_feed(self, user_id):
        """Subscribe to user's feed updates"""
        r = await redis.from_url(self.redis_url)
        pubsub = r.pubsub()

        await pubsub.subscribe(f"feed:{user_id}")

        async for message in pubsub.listen():
            if message['type'] == 'message':
                print(f"New post: {message['data']}")
```

Real-World Example: Twitter

Twitter's real-time architecture:

- **Firehose API**: Pub/Sub for real-time tweets
- **EventBus**: Internal event streaming system
- **WebSocket** for live updates
- **Timeline Service**: Fan-out to 300M+ users
- Multi-region replication for availability

Interview Tips

✅ **Do:**

- Discuss push vs pull models
- Cover connection scaling
- Mention heartbeat/keepalive
- Handle reconnection scenarios

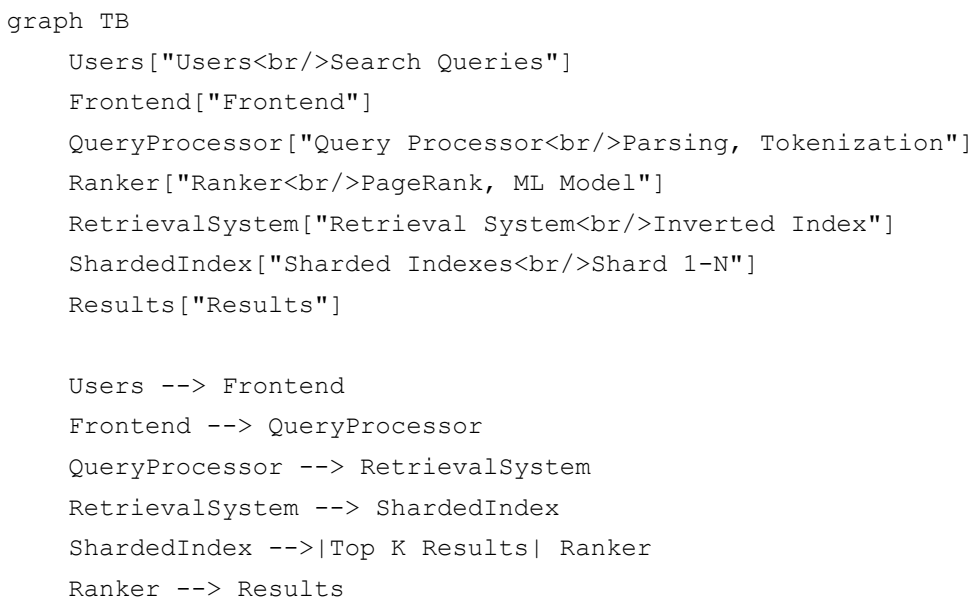
❌ **Avoid:**

- Ignoring connection management
- Missing scalability of WebSocket servers
- Assuming in-memory storage sufficient

Search Systems & Indexing

Question 9: Design Search Engine (Google-Scale)

Search System Components



Inverted Index

```
Inverted Index Structure:
---
Word: "machine"
  Document 1: Positions [5, 12, 45]
  Document 3: Positions [2, 8]
  Document 5: Positions [1]

Word: "learning"
  Document 1: Positions [6, 13]
  Document 2: Positions [3]
  Document 3: Positions [9]
```

Inverted Index Implementation

```
from collections import defaultdict, Counter

class InvertedIndex:
    def __init__(self):
        self.index = defaultdict(lambda: defaultdict(list))
        self.documents = {}

    def add_document(self, doc_id, content):
        """Index a document"""
        self.documents[doc_id] = content

        # Tokenize
        tokens = self._tokenize(content)

        # Build inverted index
        for position, token in enumerate(tokens):
            self.index[token][doc_id].append(position)

    def search(self, query):
        """Search for query"""
        tokens = self._tokenize(query)

        if not tokens:
            return []

        # AND query: intersection of documents
        result_sets = []
        for token in tokens:
            if token in self.index:
                result_sets.append(set(self.index[token].keys()))

        if not result_sets:
            return []

        # Intersect all sets
        relevant_docs = result_sets[0]
        for result_set in result_sets[1:]:
            relevant_docs &= result_set

        # Rank by BM25 or other metric
        return sorted(relevant_docs, key=self._rank, reverse=True)

    def _tokenize(self, text):
```

```
        """Simple tokenization"""
        return text.lower().split()

    def _rank(self, doc_id):
        """BM25 ranking"""
        # Simplified version
        return len(self.documents[doc_id])
```

Search Technologies Comparison

Technology	Use Case	Pros	Cons
Elasticsearch	Full-text search, logs	Easy to use, scalable	Resource intensive
Apache Lucene	Search library	Fast, flexible	Single machine
Sphinx	Full-text search	Fast indexing	Limited features
Solr	Enterprise search	Feature-rich	Complex setup

Search Pipeline: Query to Results

```
sequenceDiagram
    participant User as User
    participant QP as Query Processor
    participant Retriever as Retrieval Engine
    participant Ranker as Ranking Engine
    participant Cache as Results Cache

    User->>QP: Type query "python tutorial"
    QP->>QP: Spell check, stemming
    QP->>QP: Parse query (AND/OR/NOT)
    QP->>Retriever: Execute query
    Retriever->>Retriever: Lookup inverted index
    Retriever-->>Ranker: Top 1000 documents
    Ranker->>Ranker: BM25 + ML ranking
    Ranker-->>Cache: Store results
    Cache-->>User: Display top 10
```

Real-World Example: Google Search

Google's search system:

- **Crawler**: Indexes web pages

- **Inverted Index**: Billions of pages
- **PageRank**: Link-based ranking
- **RankBrain**: ML-based ranking signals
- **Distributed retrieval**: Thousands of shards
- **Caching**: Popular queries cached

Interview Tips

✅ **Do:**

- Understand inverted indexes
- Discuss ranking algorithms
- Cover sharding strategy
- Mention caching for popular queries

❌ **Avoid:**

- Assuming sequential search
- Ignoring ranking importance
- Missing distributed architecture

Monitoring & Observability

Question 10: Design Monitoring System for Large Scale Service

Observability Pillars

```
Metrics ——————— What is happening (aggregated)
Logs ———————————— Why it's happening (detailed)
Traces ——————— How requests flow (path)
```

Monitoring Architecture

```
graph TB
    Service["Service"]
```

```
    Metrics["Metrics<br/>Prometheus"]
    Logs["Log Aggregation<br/>ELK Stack"]
    Traces["Distributed Tracing<br/>Jaeger"]

    Alerting["Alerting<br/>AlertManager"]
    Visualization["Visualization<br/>Grafana"]

    Service -->|Emit| Metrics
    Service -->|Emit| Logs
    Service -->|Emit| Traces

    Metrics --> Visualization
    Logs --> Visualization
    Traces --> Visualization

    Metrics --> Alerting
    Alerting -->|Alert| OpsTeam["Ops Team"]
```

Key Metrics to Monitor

System Metrics:

- CPU utilization
- Memory usage
- Disk I/O
- Network bandwidth

Application Metrics:

- Request rate (QPS)
- Latency (p50, p95, p99)
- Error rate (4xx, 5xx)
- Cache hit rate
- Database query time

Business Metrics:

- Conversion rate
- Revenue per user
- Customer satisfaction

- Feature usage

Prometheus Metrics Implementation

```
from prometheus_client import Counter, Histogram, Gauge
import time

# Counter: monotonically increasing
request_count = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status']
)

# Histogram: distribution of values
request_duration = Histogram(
    'http_request_duration_seconds',
    'HTTP request latency',
    buckets=[0.01, 0.1, 0.5, 1.0, 2.0, 5.0]
)

# Gauge: current value
active_connections = Gauge(
    'active_connections',
    'Number of active connections'
)

@app.route('/api/users/<user_id>')
def get_user(user_id):
    start = time.time()

    try:
        user = fetch_user(user_id)

        request_count.labels(
            method='GET',
            endpoint='/users/{user_id}',
            status=200
        ).inc()

        return user, 200

    except Exception as e:
        request_count.labels(
            method='GET',
            endpoint='/users/{user_id}',
```

```
            status=500
        ).inc()

        return {'error': str(e)}, 500

    finally:
        duration = time.time() - start
        request_duration.observe(duration)
```

Alert Rules (for Alerting System)

```
groups:
  - name: api_alerts
    rules:
      - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High error rate detected"
          description: "Error rate is {{ $value }}%"

      - alert: HighLatency
        expr: histogram_quantile(0.99, http_request_duration_seconds) > 1
        for: 10m
        labels:
          severity: warning
        annotations:
          summary: "High p99 latency"
```

Log Aggregation Pipeline

```
Service ──> Log Shipper ──> Message Queue ──> Processing ──> Storage
               (Filebeat)      (Kafka)         (Logstash)    (Elasticsearch)
                                                                   ↓
                                                              Visualization
                                                                (Kibana)
```

Real-World Example: Netflix

Netflix's monitoring stack:

- **Metrics**: Atlas (custom, time-series database)
- **Logs**: Kafka → Suro → Elasticsearch
- **Tracing**: Trace (custom distributed tracing)
- **Alerting**: Custom alerting engine
- **Dashboard**: Custom visualization tool

Interview Tips

✅ **Do:**

- Start with key metrics (latency, error rate)
- Discuss monitoring hierarchy
- Mention alerting thresholds
- Cover tracing for debugging

❌ **Avoid:**

- Monitoring everything equally
- Ignoring alert fatigue
- Missing distributed tracing

Security & Authentication

Question 11: Design Secure Authentication System

Authentication vs Authorization

```
Authentication: WHO are you?
  ↓ Verify identity via credentials

Authorization: WHAT can you do?
  ↓ Check permissions/roles
```

Authentication Methods

Method	Security	Convenience	Use Case
Username/Password	Low	High	Basic web apps
API Keys	Medium	High	Service-to-service
OAuth 2.0	High	High	Third-party apps
JWT	High	High	Stateless APIs
SAML	High	Medium	Enterprise SSO
MFA	Highest	Low	High-security apps

OAuth 2.0 Authorization Flow

```
sequenceDiagram
    participant User
    participant App as Application
    participant AuthServer as OAuth Server
    participant ResourceServer as Resource Server

    User->>App: Login with Google
    App->>AuthServer: Redirect to authorization endpoint
    AuthServer->>User: Show consent screen
    User->>AuthServer: Grant permission
    AuthServer->>App: Return authorization code
    App->>AuthServer: Exchange code for access token
    AuthServer-->>App: Access token + Refresh token
    App->>ResourceServer: Request resource with token
    ResourceServer-->>App: Return protected resource
    App->>User: Login complete
```

JWT (JSON Web Token) Structure

```
Header.Payload.Signature

Header: {
  "alg": "HS256",
  "typ": "JWT"
}

Payload: {
  "user_id": "user_123",
  "username": "john_doe",
```

```
  "email": "john@example.com",
  "iat": 1516239022,
  "exp": 1516242622
}

Signature: HMACSHA256(base64(header) + "." + base64(payload), secret)
```

JWT Implementation

```
import jwt
import json
from datetime import datetime, timedelta
from typing import Optional

class JWTManager:
    def __init__(self, secret_key, algorithm="HS256"):
        self.secret_key = secret_key
        self.algorithm = algorithm

    def create_token(self, data: dict, expires_delta: Optional[timedelta] = None)
        """Create JWT token"""
        to_encode = data.copy()

        if expires_delta:
            expire = datetime.utcnow() + expires_delta
        else:
            expire = datetime.utcnow() + timedelta(hours=1)

        to_encode.update({"exp": expire})

        encoded_jwt = jwt.encode(
            to_encode,
            self.secret_key,
            algorithm=self.algorithm
        )
        return encoded_jwt

    def verify_token(self, token: str) -> dict:
        """Verify and decode JWT token"""
        try:
            payload = jwt.decode(
                token,
                self.secret_key,
                algorithms=[self.algorithm]
            )
```

```
            return payload
        except jwt.ExpiredSignatureError:
            raise ValueError("Token has expired")
        except jwt.InvalidTokenError:
            raise ValueError("Invalid token")

# Usage
jwt_manager = JWTManager(secret_key="your-secret-key")

# Create token
token = jwt_manager.create_token(
    data={"user_id": "user_123", "username": "john"},
    expires_delta=timedelta(hours=24)
)

# Verify token
payload = jwt_manager.verify_token(token)
print(payload)  # {'user_id': 'user_123', 'username': 'john', 'exp': ...}
```

Password Security Best Practices

```
import bcrypt
from secrets import token_urlsafe

class PasswordManager:
    @staticmethod
    def hash_password(password: str) -> str:
        """Hash password with bcrypt"""
        salt = bcrypt.gensalt(rounds=12)
        return bcrypt.hashpw(password.encode(), salt).decode()

    @staticmethod
    def verify_password(password: str, hash: str) -> bool:
        """Verify password against hash"""
        return bcrypt.checkpw(password.encode(), hash.encode())

# Best Practices:
# 1. Never store plain text passwords
# 2. Use bcrypt/scrypt/Argon2 for hashing
# 3. Salt each password uniquely
# 4. Enforce strong password policy
# 5. Implement rate limiting on login
# 6. Log authentication failures
# 7. Use HTTPS for all auth endpoints
# 8. Implement account lockout after failed attempts
```

Real-World Example: GitHub Authentication

GitHub's security:

- OAuth 2.0 for third-party app access
- Two-factor authentication (2FA)
- SSH keys for Git operations
- PATs (Personal Access Tokens) for API access
- Rate limiting on failed login attempts
- Audit logs for all actions

Interview Tips

✅ **Do:**

- Understand OAuth 2.0 flow
- Discuss JWT pros/cons
- Cover password hashing
- Mention rate limiting

❌ **Avoid:**

- Storing passwords in plain text
- Assuming authentication = authorization
- Ignoring token expiration
- Missing HTTPS

System Design Scenarios

Scenario 1: Design Twitter (Tweet Creation, Feed, Retweets)

Problem Statement

Design a system like Twitter supporting:

- 500M users, 150M DAU

- 300K tweets/second
- Real-time feed delivery
- Complex interactions (retweets, likes, replies)

Requirements

Functional:

- Users can tweet (text, images, videos)
- Users can see personalized feed
- Users can like, retweet, reply
- Trending topics

Non-Functional:

- 300K writes/second
- Sub-second feed loading
- 99.99% availability
- Global distribution

Back of Envelope Calculations

```
300K tweets/second
= 25.9B tweets/day
= 9.5T tweets/year

Storage per tweet:
- Text: 140 bytes
- Metadata: 100 bytes
- User ID: 8 bytes
= 250 bytes per tweet

Year 1: 9.5T * 250B = 2.4PB
Year 5: 12PB total storage

Peak QPS: 300K * 3 (spike) = 900K

Feed read: 150M DAU * 10 feeds/day / 86400 = 17M QPS
```

Architecture

```
graph TB
    Client["Mobile Client"]
    LB["Load Balancer"]

    TweetAPI["Tweet Service"]
    FeedAPI["Feed Service"]
    SearchAPI["Search Service"]

    Cache["Redis Cache<br/>Hot tweets, User feed"]
    TweetDB["Tweet DB<br/>Sharded"]
    UserDB["User DB"]

    MessageQueue["Message Queue<br/>Kafka"]
    FanoutWorker["Fanout Worker<br/>Push to followers"]
    SearchIndex["Search Index<br/>Elasticsearch"]

    Client -->|Write| LB
    Client -->|Read| LB

    LB --> TweetAPI
    LB --> FeedAPI
    LB --> SearchAPI

    TweetAPI -->|Write| TweetDB
    TweetAPI -->|Publish| MessageQueue
    TweetAPI -->|Cache| Cache

    FeedAPI -->|Read| Cache
    FeedAPI -->|Read| TweetDB

    SearchAPI -->|Index| SearchIndex

    MessageQueue --> FanoutWorker
    FanoutWorker -->|Push to followers| Cache
    FanoutWorker -->|Update feed| UserDB
```

Data Model

Tweets Table:

```
CREATE TABLE tweets (
    tweet_id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
```

```
    content TEXT,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    like_count INT,
    retweet_count INT,
    reply_count INT,

    INDEX (user_id, created_at),
    INDEX (created_at)
);
```

User Feed Table:

```
CREATE TABLE user_feeds (
    feed_id BIGINT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    tweet_id BIGINT NOT NULL,
    author_id BIGINT NOT NULL,
    created_at TIMESTAMP,

    INDEX (user_id, created_at)
);
```

Tweet Creation Flow

```
class TweetService:
    def create_tweet(self, user_id: str, content: str, media: list = None):
        """Create and publish tweet"""

        # Generate unique tweet ID
        tweet_id = self.snowflake_generator.generate_id()

        # Validate content
        if not self._validate_tweet(content):
            raise ValueError("Invalid tweet")

        # Save to database
        tweet = {
            'tweet_id': tweet_id,
            'user_id': user_id,
            'content': content,
            'created_at': datetime.now(),
            'like_count': 0,
            'retweet_count': 0
        }
```

```
        self.db.insert('tweets', tweet)

        # Cache tweet
        self.cache.set(f"tweet:{tweet_id}", tweet, ttl=24*3600)

        # Fanout to followers' feeds
        self.message_queue.publish('tweet_created', {
            'tweet_id': tweet_id,
            'user_id': user_id,
            'created_at': tweet['created_at']
        })

        # Update search index
        self.search_index.index_tweet(tweet)

        return tweet_id

class FanoutWorker:
    def process_tweet_created(self, event):
        """Fanout tweet to followers"""
        tweet_id = event['tweet_id']
        user_id = event['user_id']

        # Get followers
        followers = self.db.query(
            f"SELECT follower_id FROM follows WHERE following_id = {user_id}"
        )

        # Add to each follower's feed cache
        for follower_id in followers:
            feed_key = f"feed:{follower_id}"

            # Push to cache (Redis sorted set)
            score = event['created_at'].timestamp()
            self.cache.zadd(feed_key, {tweet_id: score})

            # Trim to 1000 tweets
            self.cache.zremrangebyrank(feed_key, 1001, -1)

            # Push notification
            self.notification_service.notify(
                follower_id,
                f"@{self.get_username(user_id)} tweeted"
            )
```

Feed Generation

```
class FeedService:
    def get_feed(self, user_id: str, limit: int = 50, offset: int = 0):
        """Get personalized feed for user"""

        # Try cache first
        cache_key = f"feed:{user_id}"
        cached_feed = self.cache.zrange(cache_key, offset, offset + limit - 1)

        if cached_feed:
            return self._fetch_tweet_details(cached_feed)

        # Cache miss - rebuild from database
        # Get tweets from users this person follows
        followings = self.db.query(
            f"SELECT following_id FROM follows WHERE follower_id = {user_id}"
        )

        tweets = self.db.query(f"""
            SELECT * FROM tweets
            WHERE user_id IN ({','.join(followings)})
            ORDER BY created_at DESC
            LIMIT {limit} OFFSET {offset}
        """)

        # Cache for next requests
        for tweet in tweets:
            score = tweet['created_at'].timestamp()
            self.cache.zadd(cache_key, {tweet['tweet_id']: score})

        return tweets
```

Challenges & Solutions

Challenge	Solution
Fan-out storm	Batch processing, gradual rollout
Timeline consistency	Accept eventual consistency, cache
Trending computation	Batch jobs, HyperLogLog for counts
Spam/abuse	ML detection, human review queue

Hot tweets	Separate cache, distributed caching

Follow-up Questions

Q1: How to handle trending topics?

- A: Count hashtag mentions in 1-hour windows
- Use HyperLogLog for approximate counts
- Update hourly, cache results
- Or: Publish-subscribe on hashtag updates

Q2: How to implement search?

- A: Index tweets in Elasticsearch
- Rank by relevance + recency
- Support filters: by user, date range, likes

Q3: How to support image/video tweets?

- A: Store in S3/cloud storage
- CDN for distribution
- Generate thumbnails
- Adaptive bitrate for videos

Scenario 2: Design a URL Shortening Service (Like TinyURL)

Problem Statement

Design a system to shorten long URLs:

- 100M unique URLs/month
- 10B total short URLs (scale over years)
- 100K requests/second
- High read:write ratio (100:1)

Requirements

Functional:

- Convert long URL to short code
- Redirect short URL to original
- User accounts (optional)
- Analytics (views per short URL)

Non-Functional:

- Extremely low latency (< 100ms redirect)
- 99.99% availability
- 100K QPS

Back of Envelope Calculation

```
100M URLs/month * 12 months = 1.2B URLs/year
Over 5 years: 6B URLs

Storage per URL:
- Short code: 6 bytes
- Long URL: 2000 bytes (avg)
- User ID: 8 bytes
- Creation time: 8 bytes
= ~2KB per entry

Total storage: 6B * 2KB = 12TB
Acceptable for single database

But with 100K QPS reads:
- Need distributed caching
- Multiple read replicas
```

Architecture

```
graph TB
    Client["User"]
    LB["Load Balancer"]

    ShortenerAPI["Shortener API<br/>Create short URL"]
    RedirectAPI["Redirect API<br/>Expand URL"]
    AnalyticsAPI["Analytics API"]
```

```
    Cache["Redis Cache<br/>Hot redirects"]
    PrimaryDB["Primary DB<br/>Write"]
    ReplicaDB["Replica DB<br/>Read"]

    Zookeeper["Zookeeper<br/>ID Generation"]

    Analytics["Analytics Store<br/>Kafka + Druid"]

    Client -->|POST /shorten| LB
    Client -->|GET /{short_code}| LB

    LB -->|Create| ShortenerAPI
    LB -->|Redirect| RedirectAPI
    LB -->|Analytics| AnalyticsAPI

    ShortenerAPI -->|Generate ID| Zookeeper
    ShortenerAPI -->|Write| PrimaryDB
    ShortenerAPI -->|Cache| Cache

    RedirectAPI -->|Cache lookup| Cache
    RedirectAPI -->|DB miss| ReplicaDB

    AnalyticsAPI -->|Event| Analytics
```

Unique ID Generation Strategies

Option 1: Base62 Encoding

```
Use auto-increment ID from database
Convert to base62 (0-9, a-z, A-Z)
Length: 6 characters = 62^6 = 56T possible URLs (sufficient)

Example:
ID: 123456
Base62: "B8Rc"
```

Option 2: Snowflake ID

```
Distributed unique ID generator
Timestamp (41 bits) | Machine ID (10 bits) | Sequence (12 bits)
= 64-bit unique ID
Convert to base62 for short code
```

Option 3: UUID + Hash

```
Generate UUID
Hash to 6-character string
Check for collisions (rare)
```

Implementation

```
import string
import time
from datetime import datetime
from typing import Optional

class URLShortener:
    BASE62 = string.digits + string.ascii_lowercase + string.ascii_uppercase

    def __init__(self, db, cache, id_generator):
        self.db = db
        self.cache = cache
        self.id_generator = id_generator

    def shorten_url(self, long_url: str, user_id: Optional[str] = None) -> str:
        """Create short URL"""

        # Check if already shortened
        existing = self.db.query_one(
            f"SELECT short_code FROM urls WHERE long_url = '{long_url}'"
        )
        if existing:
            return existing['short_code']

        # Generate unique ID
        unique_id = self.id_generator.generate()

        # Convert to base62
        short_code = self._int_to_base62(unique_id)

        # Store in database
        self.db.insert('urls', {
            'short_code': short_code,
            'long_url': long_url,
            'user_id': user_id,
            'created_at': datetime.now(),
            'view_count': 0
        })

        # Cache the mapping
```

```
        self.cache.set(short_code, long_url, ttl=7*24*3600)

        return short_code

    def expand_url(self, short_code: str) -> Optional[str]:
        """Expand short URL to original"""

        # Check cache first
        long_url = self.cache.get(short_code)
        if long_url:
            # Async increment view count
            self._increment_views(short_code)
            return long_url

        # Database lookup
        result = self.db.query_one(
            f"SELECT long_url FROM urls WHERE short_code = '{short_code}'"
        )

        if not result:
            return None

        long_url = result['long_url']

        # Cache it
        self.cache.set(short_code, long_url, ttl=7*24*3600)

        # Increment view count (async)
        self._increment_views(short_code)

        return long_url

    def _increment_views(self, short_code: str):
        """Increment view count"""
        # Use Redis counter for high throughput
        self.cache.incr(f"views:{short_code}")

        # Periodically flush to DB
        # Implemented in background job

    def _int_to_base62(self, num: int) -> str:
        """Convert integer to base62"""
        if num == 0:
            return self.BASE62[0]

        digits = []
        while num > 0:
```

```
            digits.append(self.BASE62[num % 62])
            num //= 62

        return ''.join(reversed(digits))

    def _base62_to_int(self, code: str) -> int:
        """Convert base62 to integer"""
        num = 0
        for char in code:
            num = num * 62 + self.BASE62.index(char)
        return num

# Usage
shortener = URLShortener(db, cache, snowflake_generator)

# Create short URL
short_code = shortener.shorten_url("https://example.com/very/long/url")
print(f"Short URL: https://short.com/{short_code}")

# Expand short URL
long_url = shortener.expand_url(short_code)
```

Database Schema

```
CREATE TABLE urls (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    short_code VARCHAR(10) UNIQUE NOT NULL,
    long_url TEXT NOT NULL,
    user_id VARCHAR(50),
    created_at TIMESTAMP,
    expires_at TIMESTAMP,
    view_count INT DEFAULT 0,

    INDEX (short_code),
    INDEX (created_at),
    INDEX (user_id)
);

CREATE TABLE analytics (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    short_code VARCHAR(10),
    user_agent TEXT,
    ip_address VARCHAR(45),
    referer TEXT,
    timestamp TIMESTAMP,
```

```
    INDEX (short_code, timestamp)
);
```

Follow-up Questions

Q1: How to handle expired URLs?

- A: Set expiration time (default 1 year)
- Cleanup job removes expired entries
- Or: Lazy deletion when accessed

Q2: How to support custom short codes?

- A: Check availability on request
- First-come, first-served basis
- Store in separate table with priority

Q3: How to prevent abuse?

- A: Rate limiting per user (10 URLs/hour)
- Detect malicious URLs (phishing, malware)
- Quarantine suspicious URLs for review

Common System Design Patterns

Pattern 1: Consistent Hashing

Problem: Adding/removing servers causes massive cache misses

Solution: Map both keys and servers to a ring, minimize movement

```
graph TB
    Key1["Key: user_1"]
    Key2["Key: user_2"]
    Key3["Key: user_3"]

    Ring["Consistent Hash Ring<br/>0 to 2^32"]

    Server1["Server 1<br/>Position: 50"]
```

```
    Server2["Server 2<br/>Position: 150"]
    Server3["Server 3<br/>Position: 250"]

    Key1 -->|Hash=10<br/>→ Server 1| Server1
    Key2 -->|Hash=200<br/>→ Server 3| Server3
    Key3 -->|Hash=80<br/>→ Server 1| Server1
```

Pattern 2: Circuit Breaker

Problem: Cascading failures when service is down

Solution: Fail fast and gracefully degrade

```
Closed (Normal)
    ↓ Error threshold exceeded
Open (Failing)
    ↓ After timeout
Half-Open (Testing)
    ↓ Success
Closed
    ↓ Failure
Open
```

Pattern 3: Bulkhead Pattern

Problem: One failing component brings down entire system

Solution: Isolate critical resources into separate pools

```
Request Pool 1: API calls (10 threads)
Request Pool 2: Database calls (20 threads)
Request Pool 3: Cache calls (15 threads)

Failure in Pool 1 doesn't affect Pool 2 or 3
```

Interview Checklist

Use this checklist for every system design question:

Requirement Clarification (5 minutes)

- Understand functional requirements

- Understand non-functional requirements
- Ask about scale (users, QPS, storage)
- Ask about latency requirements
- Ask about consistency needs

Back of Envelope Calculation (5 minutes)

- Calculate QPS
- Calculate storage needs
- Calculate bandwidth requirements
- Calculate memory needs

High-Level Architecture (10 minutes)

- Draw main components
- Show data flow
- Identify single points of failure
- Discuss scaling approach

Detailed Component Design (15 minutes)

- Design database schema
- Design API endpoints
- Design caching strategy
- Design load balancing

Scalability & Optimization (10 minutes)

- Address bottlenecks
- Discuss caching layers
- Database sharding approach
- CDN for static content

Failure Handling (5 minutes)

- Handle component failures
- Data replication strategy
- Backup and recovery
- Monitoring and alerting

Follow-up Questions (5 minutes)

- Address interviewer's concerns
- Be prepared for "what-if" scenarios
- Discuss trade-offs

Time Complexity Reference

Common Operations

Operation	Complexity
Array access	O(1)
Array search	O(n)
Hash table lookup	O(1) avg
Binary search	O(log n)
Merge sort	O(n log n)
Tree insertion	O(log n) AVL/Red-Black
Hash function	O(1) avg, O(n) worst

Network Operations

Operation	Latency
L1 cache reference	4 ns
L2 cache reference	10 ns

RAM access	100 ns
Disk random access	10 ms
Network across country	150 ms

Key Takeaways

1. **Always clarify requirements first** - Don't assume scale
2. **Think about bottlenecks early** - Database, cache, network
3. **Use proven patterns** - Caching, sharding, queues
4. **Trade-offs are key** - Speed vs cost, consistency vs availability
5. **Design for failure** - Redundancy, monitoring, fallbacks
6. **Start simple, then scale** - Monolith before microservices
7. **Communicate your thinking** - Explain why, not just what
8. **Listen to feedback** - Adjust based on interviewer's hints

---

Now you have a complete guide! Start with basics, practice with the simple concepts, then move to complex scenarios. Good luck with your system design interviews!